Actividad 4 - Diseño del prototipo.

Aurora Ortiz Jiménez

Programa de Ingeniería de Software

Corporación Universitaria Iberoamericana

Proyecto de software

Ana Mosquera

8 de noviembre 2026

Introducción

Este informe presenta el prototipo funcional del sistema de gestión de citas médicas formulado en la actividad 2. El prototipo es una versión reducida del sistema que ya funciona: permite guardar y consultar la información básica de una clínica (pacientes, médicos, especialidades, consultorios, citas e historial de atención) en una base de datos MySQL, desde un programa en Java. Un prototipo sirve para comprobar pronto si las ideas del proyecto funcionan y corregir errores antes de construir el sistema completo (Suárez, 2017).

Tipo de Prototipo

Se construyó un prototipo funcional de la parte interna del sistema (backend). No tiene pantallas: se ejecuta desde la clase principal y guarda datos reales en la base de datos. Esta decisión permitió probar primero lo más importante, que es que la información se almacene correctamente, y dejar la interfaz gráfica para una siguiente versión.

Fase 1. Recolección de Requisitos

Los requisitos se tomaron del alcance y de los criterios de aceptación del documento de formulación, y se limitaron a lo esencial para el prototipo (Tabla 1).

Tabla 1

Requisitos del prototipo

Código	Requisito	Estado
R1	Registrar pacientes con su documento, contacto y EPS	Incluido
R2	Registrar médicos asociados a una especialidad	Incluido
R3	Registrar especialidades y consultorios	Incluido
R4	Agendar citas (paciente, médico, consultorio, fecha y hora)	Incluido
R5	Registrar el historial de atención de una cita	Incluido
R6	Consultar las especialidades registradas	Incluido
R7	Evitar que dos citas se crucen en el mismo horario	Pendiente (ver refinamiento)
R8	Interfaz gráfica para el usuario	Fuera del prototipo

Fase 2. Diseño Rápido

El diseño se apoyó en el modelo relacional del curso de Bases de Datos, con seis tablas relacionadas por llaves foráneas (Tabla 2). En el programa, cada tabla tiene una clase que representa sus datos y una clase DAO que guarda y consulta esos datos. Una clase de conexión (Conexion_Data) reúne los datos para conectarse a la base clinica_citas.

Tabla 2

Tablas de la base de datos clinica_citas

Tabla	Qué guarda	Se relaciona con
especialidades	Nombre de cada especialidad médica	medicos
medicos	Datos del médico y su número de licencia	especialidades, citas
pacientes	Documento, contacto, dirección y EPS	citas
consultorios	Número, piso y sede del consultorio	citas
citas	Fecha, hora, motivo y estado de la cita	pacientes, medicos, consultorios
historial_atencion	Síntomas, diagnóstico y tratamiento de una cita	citas

Herramientas utilizadas: Java y Visual Studio Code para el programa, el conector JDBC de MySQL para comunicarse con la base de datos, y MySQL Workbench para crear las tablas y revisar los datos.

Fase 3. Construcción del Prototipo

Primero se creó la base de datos con un script SQL que incluye la creación de las tablas, datos de prueba, consultas con unión de tablas y un usuario de recepción con permisos limitados. Después se programaron en Java las clases de cada tabla y sus DAO, que usan consultas preparadas para guardar la información de forma segura. La clase App.java se usó para ejecutar cada prueba de registro. Las Figuras 1 a 6 muestran el contenido de cada tabla después de las pruebas.

Figura 1

Tabla especialidades

```
1 •  SELECT * FROM clinica_citas.especialidades;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Co

id_especialidad	nombre_especialidad
1	Medicina General
2	Pediatría
3	Cardiología
NULL	NULL

Nota. Captura de MySQL Workbench con tres especialidades registradas.

Figura 2

Tabla consultorios

Limit to 1000 rows

```
1 •  SELECT * FROM clinica_citas.consultorios;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Conte

id_consultorio	numero	piso	sede
1	101	1	Sede Norte
2	205	2	Sede Norte
3	1	105	Sede Norte
NULL	NULL	NULL	NULL

Nota. Captura de MySQL Workbench. El consultorio 3 se registró desde Java.

Figura 3

Tabla citas

```
1 •  SELECT * FROM clinica_citas.citas;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_cita	id_paciente	id_medico	id_consultorio	fecha	hora_inicio	hora_fin	motivo_consulta	estado
1	1	1	1	2026-10-05	09:00:00	09:30:00	Consulta general de control	Atendida
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Nota. Captura de MySQL Workbench. La cita de prueba aparece con estado Atendida.

Figura 4

Tabla medicos

```
1 •  SELECT * FROM clinica_citas.medicos;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_medico	nombres	apellidos	numero_licencia	telefono	correo	id_especialidad
1	Laura	Gómez Rueda	MED-1023	3011234567	laura.gomez@clinica.com	1
2	Andrés	Pineda Castro	MED-1045	3017654321	andres.pineda@clinica.com	3
7	Camila	Villa	15205890	3207089969	camila.villa@medicos.com	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Nota. Captura de MySQL Workbench. La médica Camila Villa se registró desde Java.

Figura 5

Tabla pacientes

```
1 •  SELECT * FROM clinica_citas.pacientes;
```

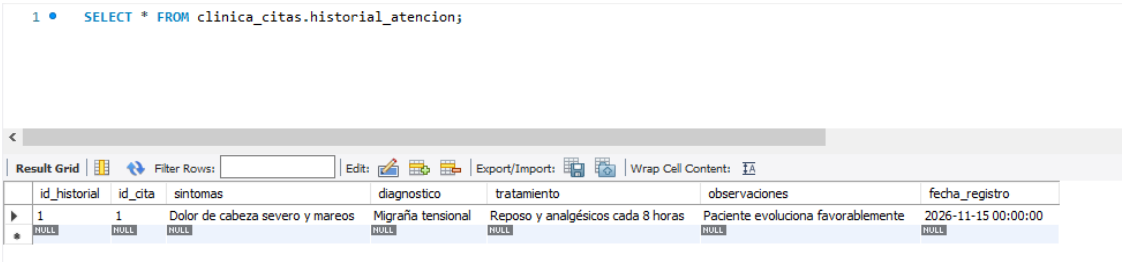
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_paciente	tipo_documento	numero_documento	nombres	apellidos	fecha_nacimiento	telefono	correo	direccion	eps
1	CC	1017234567	Mariana	Torres Vélez	1994-03-12	3159876543	mariana.torres@correo.com	Cra 45 #12-30	Sura
2	Cédula	1004626436	Juan	Villanueva	NULL	3215627895	juan.villanueva@gmail.com	Dosquebradas - Buenos Aires Cra: #24-54	Nueva Esp
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Nota. Captura de MySQL Workbench. El paciente Juan Villanueva se registró desde Java.

Figura 6

Tabla historial_atencion



Nota. Captura de MySQL Workbench. El registro se guardó desde Java y quedó ligado a la cita 1.

Fase 4: Evaluación

Se probó cada función ejecutando App.java y revisando después la tabla en MySQL. Durante las pruebas aparecieron varios errores, que se resumen en la Tabla 3. Encontrarlos a tiempo es el propósito del prototipo.

Tabla 3

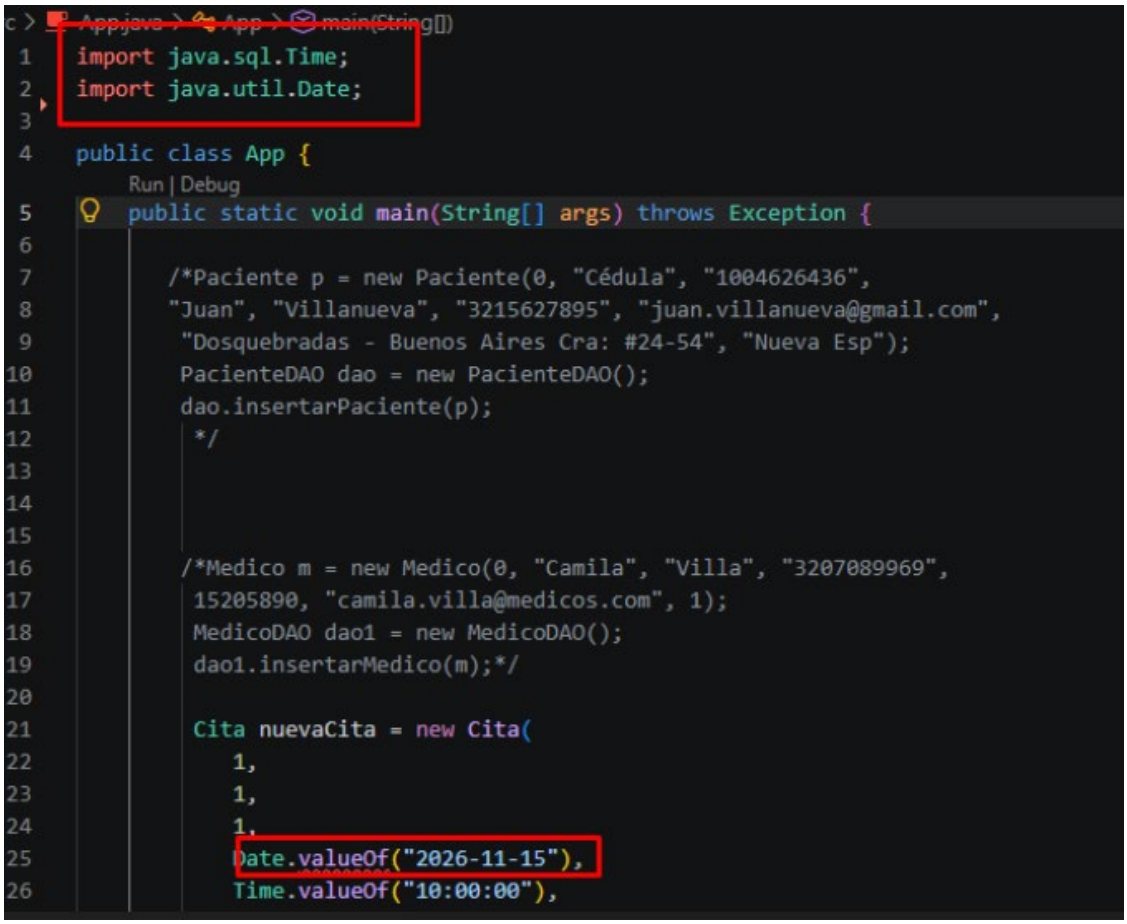
Pruebas realizadas y resultados

Prueba	Resultado	Causa
Conexión a la base de datos	Exitosa	No aplica
Registrar consultorio	Error	El programa usaba un nombre de columna que no existe (numero_consultorio)
Registrar paciente	Error, luego exitosa	El programa usaba una columna llamada nombre; la tabla tiene nombres
Registrar médico	Error	Se intentó usar una especialidad que aún no existía (llave foránea)
Ejecutar App.java con fecha	Error de compilación	Se importó java.util.Date en lugar de java.sql.Date
Registrar historial de atención	Falló la primera vez	La tabla no tenía la columna sintomas
Revisión de cada tabla con SELECT	Exitosa	Los datos se guardaron correctamente
Agendar cita desde Java (CitaDAO)	Exitosa	Los datos se guardaron correctamente

Nota. Elaboración propia a partir de los mensajes de error y las capturas de la ejecución.

Figura 7

Error de importación en App.java



Nota. Captura de Visual Studio Code. Los recuadros rojos marcan la importación y el uso de Date.valueOf.

Al revisar las capturas también se vio que el consultorio 3 quedó con número 1 y piso 105, porque los datos se enviaron en orden invertido. Además, el método para agendar citas solo guarda la cita y todavía no revisa si el médico ya está ocupado a esa hora.

Fase 5: Refinamiento

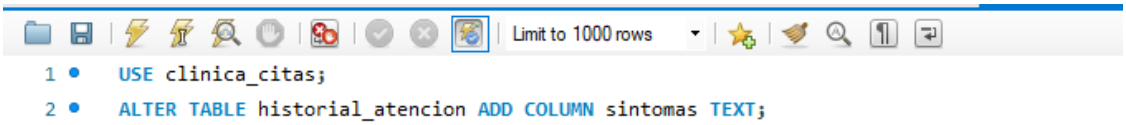
Con base en los resultados de la evaluación se hicieron estos ajustes:

- Se corrigieron los nombres de las columnas en los DAO de consultorios y pacientes para que coincidan con las tablas.
- Se registraron los datos en el orden correcto: primero las especialidades y después los médicos.

- Se cambió la importación a java.sql.Date para que la fecha funcione con la base de datos.
- Se agregó la columna sintomas a la tabla historial_atencion (Figura 8), y la columna eps a pacientes.

Figura 8

Columna sintomas agregada a historial_atencion



Nota. Captura de MySQL Workbench con la instrucción ALTER TABLE ejecutada.

Mejoras para la siguiente versión:

- Revisar antes de guardar una cita que el médico no tenga otra en el mismo horario (requisito R7).
- Corregir el registro del consultorio 3 y revisar el orden de los datos al crear consultorios.
- Cambiar el número de licencia del médico a texto en la clase Medico, porque en la tabla se guarda como texto (por ejemplo, MED-1023).
- Sacar el usuario y la contraseña de la base de datos del código y guardarlos en un archivo de configuración.
- Crear un menú o pantallas para que un usuario pueda usar el sistema sin editar el código.

Conclusiones

El prototipo cumple su objetivo principal: el programa en Java se conecta a MySQL y guarda correctamente la información de pacientes, médicos, especialidades, consultorios, citas e historial de atención. Seguir el ciclo de vida del prototipo ayudó a encontrar errores pequeños desde el inicio, como nombres de columnas distintos o datos registrados en desorden, y a corregirlos antes de crecer el proyecto. Queda pendiente validar que las citas no se crucen, que era un criterio de aceptación del proyecto, y construir una interfaz para el usuario final.

Referencias

Gual Ortí, J. (2016). Fundamentos del modelado y prototipado virtual en el diseño de productos. Universitat Jaume I, Servei de Comunicació i Publicacions.

Suárez, E. C. (2017). Prototipo, contexto e ingeniería del software. Estudios de Postgrado en Sistemas de Información, 2–20.